INSA TOULOUSE

Virtualisation et Cloud Computing

2026–2027

Evelyne AKOPYAN
evelyne.akopyan@insa-toulouse.fr

TP VCC-3

Déployer une application distribuée

Objectifs pédagogiques

À l'issue de cette séance, vous devrez être capables de :

- concevoir une architecture distribuée adaptée à une application composée de plusieurs microservices
- répartir les services sur plusieurs machines virtuelles OpenStack
- mettre en place une architecture réseau permettant leur communication
- assurer le bon fonctionnement de l'ensemble de la plateforme
- préparer votre infrastructure en vue de son automatisation.

1 Concevoir l'architecture de la plateforme

Objectif

Concevoir une architecture distribuée permettant de déployer l'application Calculator dans le Cloud.

Votre startup souhaite désormais mettre en production son application **Calculator**.

Jusqu'à présent, toutes les applications manipulées étaient exécutées sur une unique machine virtuelle. Cette approche devient rapidement limitée lorsqu'une application est composée de plusieurs services indépendants.

1.1 Contraintes à respecter

Votre architecture devra respecter les contraintes suivantes :

- chaque service devra être exécuté dans un **conteneur Docker**
- une machine virtuelle ne peut faire tourner qu'un seul conteneur à la fois
- les services devront pouvoir communiquer entre eux
- seuls les services destinés aux utilisateurs devront être accessibles depuis Internet
- les services internes devront rester isolés dans un réseau privé.

1.2 Ressources disponibles

Vous disposez des ressources suivantes :

- VM : 6 maximum
- Image : ubuntu-24.04
- Flavors : insa-tiny, insa-small et insa-medium
- CPU : 6 maximum
- RAM : 6 Go maximum
- Réseau virtuel : 1
- Routeur : "routerdefault"
- Les images Docker construites lors du TP1

A partir de ces ressources, réalisez un schéma représentant votre future plateforme. Vous pouvez utiliser l'outil de votre choix (par exemple draw.io). Votre schéma devra faire apparaître :

- les VM + flavor
- les conteneurs Docker + services associés
- le.s réseau.x OpenStack + routeur
- les principales communications entre les services + ports
- l'accès depuis Internet.

1.3 Checkpoint

Avant de poursuivre le TP, présentez votre proposition à l'enseignant. Une fois votre architecture validée, dans le fichier "TP3/README.md", ajoutez une section "Etape 1" et importez-y votre schéma d'architecture.

2 Déployer les services

Objectif

Déployer les microservices de l'application Calculator sur l'architecture conçue à l'étape précédente.

2.1 Création des VM

Créez les différentes machines virtuelles prévues dans votre architecture. Vous pouvez utiliser l'interface Horizon ou la CLI, au choix. Pensez à donner à vos VM des noms explicites.

Une fois les machines créées, vérifiez qu'elles sont toutes démarrées, et attribuez une IP flottante aux machines qui doivent être accessibles depuis Internet.

2.2 Accès SSH

Vous allez peut-être avoir plus de VM que d'IP flottantes, et c'est normal : afin d'accéder aux VM du réseau privé, vous devez d'abord passer par la VM connectée à Internet, qui servira de **proxy réseau**.

Pour configurer les accès SSH :

— Sur votre machine hôte, importez la clé privée de toutes les VM dans le dossier ~/.ssh/
— Configurez SSH pour le multi-saut dans le fichier **config** (à modifier/créer dans le dossier) :

```
Host <vm-proxy>
    HostName <IP_FLOTTANTE>
    User ubuntu
    IdentityFile ~/.ssh/<KEYNAME>.pem

Host <vm-privee-1>
    HostName <IP_PRIVEE>
    User ubuntu
    IdentityFile ~/.ssh/<KEYNAME>.pem
    ProxyJump <vm-proxy>

Host <vm-privee-2>
    HostName <IP_PRIVEE>
    User ubuntu
    IdentityFile ~/.ssh/<KEYNAME>.pem
    ProxyJump <vm-proxy>

...
```

Si vous êtes sur Windows, pensez à vérifier les permissions système de lire vos clés SSH :

```
icacls "$env:USERPROFILE\.ssh\<KEY>.pem" /inheritance:r
icacls "$env:USERPROFILE\.ssh\<KEY>.pem" /grant:r "${env:USERNAME}:(R)"
```

Une fois le proxy et SSH configurés, accédez à vos VM en adaptant la commande avec vos noms de VM tels que déclarés dans le fichier config :

```
ssh <vm-proxy>
ssh <vm-privee>
```

2.3 Déploiement des services

Récupérez le dépôt utilisé lors du TP1 sur chacune de vos VM :

```
git clone https://github.com/eakopyan/insa-labs
```

Et placez-vous dans le dossier **Calculator**. Déployez ensuite les services conformément à votre architecture. Vérifiez ensuite que les conteneurs sont correctement démarrés.

2.4 Checkpoint

Dans le fichier "TP3/README.md", ajoutez une nouvelle section "Étape 2" et répondez aux questions suivantes :

1. Quels sont les différents rôles de la VM connectée à Internet dans votre architecture ?
2. Quels avantages apporte ce déploiement par rapport à une exécution de tous les services sur une seule machine ?

3 Etablir les communications

Objectif

Mettre en place une architecture réseau permettant aux différents microservices de communiquer entre eux.

3.1 Règles de sécurité

Observez le code de chaque service et notez les protocoles et ports utilisés pour la communication (en interne, et avec l'extérieur). Ajustez les règles de sécurité de votre Groupe de Sécurité si nécessaire.



Important : Si certaines règles ne s'appliquent qu'aux communications internes, pensez à les limiter aux "Groupes de Sécurité" et non au CIDR : cela autorisera le trafic uniquement entre les VM appartenant au même groupe de sécurité.

3.2 Vérification des communications

Procédez progressivement. **Ne cherchez pas à tester toute l'application d'un seul coup.**

Commencez par vérifier que chaque machine est joignable depuis les autres (ping).

Vérifiez ensuite que chaque service est bien disponible sur le port associé :

```
curl http://localhost:<PORT>
```

Puis faites le même test depuis une autre VM afin de vérifier que le service soit accessible sur le réseau privé :

```
curl http://<IP_PRIVEE>:<PORT>
```

Ensuite seulement, faites le test de l'application complète en contactant le service frontend connecté à Internet. Depuis votre machine hôte :

```
curl -v -X POST -d "2+3" http://<IP_FLOTTANTE>:<PORT>
```

3.3 Checkpoint

Dans le fichier "TP3/README.md", ajoutez une section "Étape 3" et répondez aux questions suivantes :

1. Quels flux réseau sont nécessaires au fonctionnement de votre application ?
2. Pourquoi est-il préférable d'ouvrir uniquement les ports nécessaires plutôt que d'autoriser tout le trafic ?
3. Pouviez-vous utiliser Docker Compose pour déployer les services sur toutes les VM en simultané ? Pourquoi ?

4 Pour aller plus loin : améliorer la sécurité réseau

Votre entreprise décide de renforcer la sécurité réseau de votre application. Pour cela, il est proposé d'**isoler** le service frontend dans un réseau privé à part, connecté à Internet.

4.1 Topologie cible

Créez un deuxième réseau privé (192.168.10.0/24) connecté au routeur par défaut, et placez-y le service frontend (calc-service).

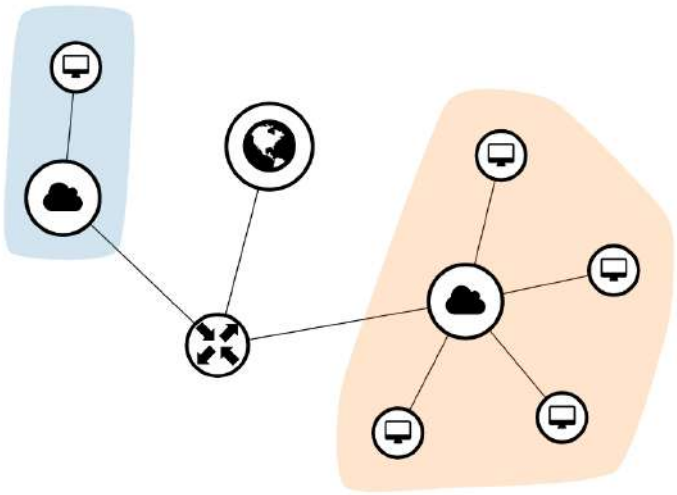


Figure 1 – Topologie cible

4.2 Règles de routage

En l'état, votre service frontend ne peut plus accéder aux autres services. Pour y remédier, depuis la VM du frontend, ajoutez des règles de routage pour rétablir le trafic :

```
sudo route add -net <DEST_NETWORK>/<MASK> gw <GW_ADDRESS>
```

Puis testez à nouveau la connectivité comme à l'étape précédente.